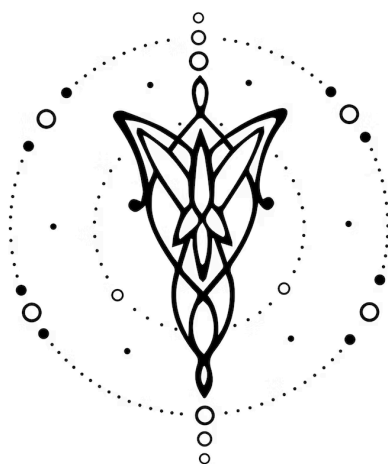




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

V1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Alexis	Herrera Vegas	64045	aherreravegas@itba.edu.ar
Jeronimo	Esquivel	64756	tesquivel@itba.edu.ar
Andres	Cortese	64612	acortese@itba.edu.ar
Sebastian	Caules	64331	scaules@itba.edu.ar

2. Repositorio

[Link del repositorio](#)

3. Dominio

Desarrollar un lenguaje que permita crear una agenda con la planificación de eventos en un calendario, que sea representado de forma gráfica y acorde al formato común de calendario. El lenguaje debe permitir crear eventos para días y horarios determinados, estos pueden ser recurrentes si se quiere repetir una tarea en determinado día por un tiempo determinado o indeterminado.

El lenguaje transforma la declaración de un evento en un formato simple similar a un JSON o un struct de C en código HTML que representa gráficamente el calendario con los estilos (colores) elegidos. Los eventos se declaran en estructuras formadas por pares claves : valor, y permiten elegir día, horario, color, nombre, etc. Además es posible gestionar eventos ya existentes, al borrarlos o reemplazarlos.

La implementación satisfactoria de este lenguaje permitiría organizar y visualizar fácilmente los eventos que debe afrontar una persona en su día a día. Así se reemplazaría definitivamente el uso de agendas físicas o el uso para este propósito de softwares de hojas de cálculo como Excel, que suelen ser utilizadas pero requieren un mayor tiempo y esfuerzo, aparte de no estar preparadas específicamente para esto. El lenguaje presenta una tal simpleza que incluso puede ser aprovechada por otros softwares para representar visualmente eventos en un calendario.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

- (I). Se podrán crear calendarios especificando el año y el mes, aceptando tanto nombres de meses en inglés o valores numéricos.
- (II). Se podrán declarar eventos particulares mediante bloques `Event {}`,
 - (II.1). El tag `title` declara el título del evento y es de tipo string, sin necesidad de usar comillas
 - (II.2). El tag `date` declara la fecha del evento y es de tipo int en donde el número n debe cumplir: $n \in \mathbb{N}_0$
 - (II.3). Los tags `start` y `end` declaran el inicio y fin del evento y es de tipo string, sin necesidad de usar comillas y con el formato específico `hh:mm`
 - (II.4). El tag `color` es opcional y declara el color del evento. El tipo del color puede ser hexadecimal o se puede elegir un color de los 16 defaults.
 - (II.5). El tag `replace` es opcional y reemplaza un evento con el mismo horario previamente declarado por el evento que se declarara a continuación. Al usarlo se deben eliminar los tags `start` y `end` ya que adopta el horario del evento a reemplazar.

- (II.5.1). Con el tag `declare` seguido de un nombre sin conflictos se puede declarar un color particular (por ejemplo, `declare cyan: #00ffff`)
- (III). Se podrán declarar eventos recurrentes sobre un día de la semana (por ejemplo: `Monday {}`), los cuales se repetirán automáticamente en todas sus ocurrencias dentro del mes. Todos los tags expuestos previamente deben estar presentes en esta declaración.
 - (III.1). El tag `which` reemplaza el tag `date` previamente explicado y declara la recurrencia del evento y es de tipo string sin comillas pero solo se pueden usar las palabras claves *first*, *last*, *any*, *odd* o *even*.
- (IV). Se podrán concatenar declaraciones de días para la evitar la redundancia de código en el caso anterior (por ejemplo: `Monday, Friday {}`)
- (V). Se aceptarán comentarios en el código precedidos por `//`, los cuales no se verán reflejados en la salida.

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que declare un evento en una fecha particular
- (II). Un programa que permita declarar un evento en un día de la semana y lo repita automáticamente en todas sus ocurrencias dentro del mismo mes.
- (III). Un programa que permita declarar un evento en un día de la semana y lo repita bisemanalmente
- (IV). Un programa que permita declarar dos eventos en un mismo día y/o mismo horario
- (V). Un programa que use un color declarado con `declare`.
- (VI). Un programa que defina un evento recurrente en varios días concatenados.
- (VII). Un programa que declare un evento recurrente con `which: first` o `which: last`.
- (VIII). Un programa que reemplace un evento recurrente en una fecha específica con `replace`.
- (IX). Un programa que acepte distintas formas de escribir el mes y genere el mismo calendario.
- (X). Un programa que permita una combinación de cualquiera de los casos anteriores

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado.
- (II). Un programa que declare con un color para el `color` el cual no se haya declarado previamente o que no sea un color default.
- (III). Un programa que asigne un evento en una fecha inexistente como 30 de febrero o 32 de cualquier mes.
- (IV). Un programa que declare un horario de fin `end` previo al horario de inicio `start`
- (V). Un programa que declare un `title` de más de 64 caracteres

- (VI). Un programa que declara un color con un nombre utilizado por el lenguaje (por ejemplo, declare Monday)
- (VII). Un programa el cual en primera instancia declare un evento recurrente x y luego en otro evento se quiera reemplazar a x con un evento con una date la cual no matchee a ninguno de los días de la recurrencia de x.
Ejemplo, evento recurrente x los lunes de agosto del 2025 y el nuevo evento quiere reemplazar a x y declara date: 10 (domingo)
- (VIII). Un programa en donde se declare un evento que quiera reemplazar a otro evento el cual no se declaró previamente, esto incluye cualquier error de sintaxis del string.

6. Ejemplos

Creación de un calendario para el mes de **agosto de 2025**. Se agrega un evento recurrente los martes de 10:30 a 11:30 hs llamado **GYM** en color celeste y otro evento recurrente los lunes de 14:00 a 16:00 hs llamado **Proba** en color rojo, que en su ocurrencia del día 11 se reemplaza por **Parcialito proba** en color cyan.

```
Year: 2025
Month: August

declare cyan: #00FFFF

Tuesday {
  which: any           // Todos los lunes del mes
  title: GYM
  start: 11:30
  end: 12:30
  color: light_blue
}

Monday {
  which: any           // Todos los lunes del mes
  title: Proba
  start: 14:00
  end: 16:00
  color: red
}

Event {
  date: 11
  replace: Proba
  title: Parcialito proba
  color: cyan
}
```

Creación de un calendario para el mes de **octubre de 2025**. Se incorpora un evento recurrente los lunes y viernes de 08:00 a 10:00 hs llamado **Taller** en color azul para sus ocurrencias impares, un evento recurrente los miércoles de 14:00 a 16:00 hs llamado **Seminario Redes** en color magenta, y un evento recurrente los martes y jueves de 17:00 a 18:00 hs llamado **Consultas** en color cobalt, donde la iteración del día 17 reemplaza **Taller** por **Taller especial** en color lime.

```
Year: 2025
Month: 10 // Mes numérico permitido

declare cobalt:#0047AB // Color declarado (no default)
declare lime:#32CD32 // Otro color declarado

Monday, Friday {
  which: odd // Ocurrencias impares del mes (1ª, 3ª, 5ª)
  title: Taller // title es string sin comillas
  start: 08:00 // hh:mm sin comillas
  end: 10:00
  color: blue // color default
}

Wednesday {
  which: first // Primera ocurrencia del mes
  title: Seminario Redes
  start: 14:00
  end: 16:00
  color: #FF00FF // color en hexadecimal
}

Tuesday, Thursday {
  which: even // Ocurrencias pares del mes (2ª, 4ª, ...)
  title: Consultas
  start: 17:00
  end: 18:00
  color: cobalt // uso de color declarado
}

// Reemplazo de una ocurrencia existente de "Taller":
// adopta el horario del evento reemplazado, por eso NO se indican start/end.
Event {
  date: 17 // 17/10/2025 es viernes (coincide con "Friday which: odd")
  replace: Taller
  title: Taller especial
  color: lime // uso de color declarado
}
```

2025		OCTOBER					MONDAY	
CALENDAR YEAR							FIRST DAY OF WEEK	
Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday		
29	30	01 Seminario Redes 14:00-16:00	02	03 Taller 08:00-10:00	04	05		
06 Taller 08:00-10:00	07	08	09 Consultas 17:00-18:00	10	11	12		
13	14 Consultas 17:00-18:00	15	16	17 Taller especial 08:00-10:00	18	19		
20 Taller 08:00-10:00	21	22	23 Consultas 17:00-18:00	24	25	26		
27	28 Consultas 17:00-18:00	29	30	31 Taller 08:00-10:00	01	02		

Ejemplo del output del ejemplo 2 en formato html hecho con un programa de prueba en python